département
Informatique

Université Claude Bernard Lyon 1

Année universitaire	**2026-2027**		
Filière	**Informatique**	**Année**	**M2 DS**
Module	**Machine Learning**		
Enseignants	**Haytham Elghazel et Heidi Souibki**		
Intitulé :	**Atelier : Détection d'anomalies avec Python**		
Contenu	• **Détection d'anomalies : outliers/nouveautés** • **Approches non supervisées : Isolation Forest, LOF** • **Approches supervisées pour la détection d'anomalie** • **Apprentissage sur des données déséquilibrées (Imbalanced Learning)** • **Évaluation en présence de données déséquilibrées**		
Rendu	**Rendu sur Moodle : Un code factorisé, bien structuré et commenté**		

Dans cet atelier pratique, Vous allez comparer des méthodes de détection d'anomalies, construire un protocole d'évaluation et choisir un seuil répondant à une contrainte opérationnelle. Vous devrez expliquer vos résultats et rechercher une méthode non étudiée en cours. Toutes les parties sont obligatoires.

Organisez le travail dans un notebook principal et un fichier utils.py regroupant les fonctions réutilisables de préparation, apprentissage, calcul des scores, choix du seuil et évaluation. Fixez les graines aléatoires, indiquez les versions des bibliothèques et mesurez les temps d'apprentissage et de calcul des scores. Ne masquez pas systématiquement les avertissements.

Bibliothèques utiles : numpy, pandas, matplotlib ou seaborn, scikit-learn, imbalanced-learn et SHAP. Ajoutez uniquement les dépendances nécessaires à la méthode que vous aurez recherchée.

Jeux de données

- **Mouse** : fichier mouse.txt fourni avec le TP, issu de la collection ELKI. Il contient 500 points décrits par deux variables. Le fichier ne contient pas de variable cible. Sachant que les dix derniers points sont les anomalies de référence, construisez un vecteur d'étiquettes destiné uniquement à l'évaluation. Construisez-le avant de mélanger ou de partitionner les données, puis conservez l'alignement entre les observations et leurs étiquettes.
- **Credit Card Fraud Detection** : 284 807 transactions, dont 492 fraudes. Utilisez le fichier creditcard.csv. Source : https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud
- **AI4I 2020 Predictive Maintenance** : 10 000 observations synthétiques de maintenance industrielle. Source UCI : https://archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset

La détection d'outliers recherche des observations atypiques dans un ensemble pouvant être contaminé. **La détection de nouveautés** apprend une référence à partir d'observations normales, puis évalue de nouvelles observations. Une anomalie statistique n'est pas nécessairement une fraude ou une panne.

Protocole commun et mesures

1. **Séparer apprentissage et évaluation**
   - Pour Credit Card et AI4I, utilisez un découpage stratifié reproductible de 60 % pour l'apprentissage, 20 % pour la validation et 20 % pour le test. Réutilisez exactement les mêmes partitions pour toutes les méthodes d'un même jeu. Ce découpage constitue ici un cadre de comparaison. Yne utilisation prospective demanderait aussi d'étudier un découpage temporel.
   - Ajustez les transformations sur l'apprentissage uniquement. Appliquez tout rééchantillonnage uniquement à cette partition. La validation sert au choix des hyperparamètres, de la méthode et du

seuil. Le test reste réservé à l'évaluation finale. Ne rééquilibrez ni la validation ni le test. La validation croisée n'est pas exigée.

- Limitez la recherche à six configurations par famille de modèles. Pour les méthodes coûteuses, utilisez un sous-échantillon stratifié fixe de l'apprentissage, d'au plus 30 000 observations, commun aux comparaisons concernées. Gardez les partitions de validation et de test inchangées et indiquez les effectifs réellement utilisés.

2. **Deux niveaux d'évaluation**
   - **Codez la classe d'intérêt par 1 :** anomalie, fraude ou panne. Harmonisez les scores pour qu'une valeur élevée indique une observation plus suspecte. Les scores d'Isolation Forest dans scikit-learn sont orientés dans le sens inverse : vérifiez leur signe avant l'évaluation.
   - **Évaluation du classement :** tracez les courbes ROC et précision-rappel à partir des scores continus. Calculez l'AUC-ROC et l'Average Precision (AP) avec average_precision_score(y, scores). L'AP est la mesure principale de classement : elle résume la relation entre rappel des anomalies et précision des alertes.
   - **Évaluation après choix du seuil :** fournissez la matrice de confusion, la précision, le rappel, le F1 de la classe positive et la balanced accuracy. Le F1 combine précision et rappel. La balanced accuracy est la moyenne des rappels des deux classes ; elle ne mesure pas directement la fiabilité des alertes.
3. **Objectifs opérationnels imposés**
   - **Fraude :** maximiser le rappel sous la contrainte précision ≥ 80 %. Le scénario suppose une équipe qui veut retrouver le plus de fraudes possible tout en limitant les enquêtes inutiles : sur la validation, au moins quatre alertes sur cinq doivent correspondre à des fraudes.
   - **Maintenance :** maximiser le rappel sous la contrainte précision ≥ 90 %. Le scénario suppose que chaque alerte entraîne une intervention coûteuse : la fiabilité des alertes doit être plus élevée. Ces deux valeurs sont des contraintes pédagogiques, pas des normes universelles.
   - Pour chaque configuration, cherchez sur la validation le seuil donnant le meilleur rappel parmi ceux qui satisfont la contrainte et produisent au moins une alerte. En cas d'égalité, privilégiez la précision, puis l'AP. Signalez le nombre d'alertes soutenant le résultat : une précision élevée sur très peu de cas est fragile.
   - Si aucune configuration ne satisfait la contrainte, indiquez « objectif non atteint » et présentez le meilleur compromis observé sans abaisser silencieusement l'exigence. Figez ensuite le modèle et le seuil avant le test. Rapportez la précision et le rappel obtenus sur le test, même si la contrainte n'y est plus respectée.

Partie 1 : Données Mouse

1. **Observer et comparer**
   a. Chargez les données, vérifiez leur format et représentez les points dans le plan. Conservez les étiquettes pour l'évaluation. Ne les utilisez pas pour ajuster les méthodes non supervisées.

   b. Appliquez Isolation Forest et LOF à l'ensemble Mouse pour détecter ses outliers. Affichez, pour chaque méthode, les points détectés, une représentation colorée par score et l'histogramme des scores. Pour LOF, distinguez le score disponible sur les observations d'apprentissage de celui utilisé ensuite en détection de nouveautés.

2. **Choisir un seuil sans étiquettes**
   a. Construisez une règle de seuillage des scores fondée sur l'écart interquartile. Cette règle est une heuristique et donc ne garantit pas un meilleur seuil.

   b. Comparez ces règles au seuil par défaut de chaque méthode. Fournissez les points détectés, la matrice de confusion et le F1 de la classe anomalie. Le F1 est ici la mesure principale : il pénalise à la fois les anomalies manquées et les points normaux signalés à tort. Ne choisissez pas les paramètres des heuristiques en consultant les étiquettes.

c. Calculez l'AP et l'AUC-ROC des scores. Expliquez pourquoi modifier seulement le seuil ne modifie pas ces deux mesures. Ces résultats sur Mouse décrivent la détection dans l'échantillon observé ; ils ne constituent pas une estimation sur des données futures.

3. **Passer à la détection de nouveautés**
   a. Réservez une partie des points normaux et toutes les anomalies pour un test distinct. Ajustez LOF avec novelty=True sur les seuls points normaux restants, puis évaluez uniquement les points réservés. Présentez séparément les observations d'apprentissage et de test sur une figure.

   b. Expliquez ce qui change entre la détection d'outliers dans un ensemble et la détection de nouveautés. Indiquez pourquoi l'utilisation des étiquettes pour constituer un apprentissage normal modifie les informations disponibles.

Résultats attendus
Des figures lisibles, un tableau comparatif des règles de seuil et une courte analyse des désaccords entre IF et LOF. Discutez la fragilité des résultats lorsque seules dix anomalies sont disponibles.

Partie 2 : Détection des fraudes

1. **Préparer Credit Card**
   a. Examinez les types, les valeurs manquantes, les doublons éventuels et la proportion de fraudes. Évitez qu'une transaction dupliquée se retrouve dans plusieurs partitions. Écartez Time des variables prédictives pour cette comparaison ; conservez-la pour discuter les limites du découpage aléatoire.

   b. Étudiez les échelles des variables, notamment Amount. Justifiez le prétraitement choisi pour chaque famille : une méthode fondée sur les distances est particulièrement sensible à l'échelle. Les variables V1 à V28 étant anonymisées par transformation, leur interprétation métier est limitée.

2. **Comparer et prendre une décision**
   a. Comparez deux modèles supervisés : un arbre de décision et une régression logistique. Pour chaque modèle, étudiez trois variantes : sans correction du déséquilibre, avec pondération des classes et avec une méthode de rééchantillonnage choisie et justifiée parmi RandomUnderSampler, RandomOverSampler, SMOTE ou Tomek Links. Appliquez le rééchantillonnage uniquement aux données d'apprentissage.

   b. Pour chaque modèle supervisé, testez au maximum **six configurations, toutes variantes comprises**. Une configuration correspond à une combinaison d'hyperparamètres et de traitement du déséquilibre. Par exemple, tester deux profondeurs d'arbre pour chacune des trois variantes représente six configurations. Justifiez vos choix et sélectionnez la meilleure configuration sur l'ensemble de validation.

      Tomek Links réalise un nettoyage des observations proches de la frontière entre les classes, sans garantir l'équilibrage des effectifs. Pour SMOTE, veillez notamment à l'échelle des variables utilisées pour calculer les voisinages. Les prétraitements doivent être ajustés uniquement sur les données d'apprentissage.

   c. Ajoutez deux détecteurs non supervisés : Isolation Forest et LOF. Ajustez-les sur les données d'apprentissage contenant des transactions normales et frauduleuses, sans utiliser leurs étiquettes ni leur appliquer de rééchantillonnage supervisé. Pour LOF, utilisez novelty=True afin de calculer les scores des observations de validation et de test. N'utilisez pas ce mode pour réévaluer les observations ayant servi à son apprentissage.

   d. Distinguez l'ajustement du détecteur de la sélection de ses réglages : Isolation Forest et LOF sont ajustés sans étiquettes, mais le choix de leurs hyperparamètres et de leur seuil à partir des étiquettes de validation constitue une étape de sélection supervisée. Expliquez cette distinction dans votre analyse et réservez les étiquettes du test à l'évaluation finale.

3. **Évaluer les scores et fixer le seuil**

a. Sur la validation, comparez les AP et AUC-ROC. Tracez les courbes ROC et précision-rappel. Pour un même détecteur, représentez le F1 et la balanced accuracy en fonction du seuil. Expliquez pourquoi leurs maxima ne coïncident pas nécessairement avec l'objectif opérationnel.

b. Appliquez la règle imposée : maximiser le rappel sous précision ≥ 80 %. Comparez le seuil retenu au seuil par défaut. Sélectionnez vos configurations et votre méthode avant toute consultation des résultats du test.

c. Évaluez ensuite les configurations retenues sur le test. Le tableau final doit contenir : méthode, prétraitement, traitement du déséquilibre, AP, AUC-ROC, seuil retenu, précision, rappel, F1, balanced accuracy, nombres de fausses alertes et de fraudes manquées, temps de calcul. Identifiez la méthode sélectionnée sur la validation.

d. Interprétez les résultats en nombres de transactions : combien d'alertes faut-il examiner et combien de fraudes échappent à la détection ? Si le modèle sélectionné échoue à respecter la contrainte sur le test, expliquez ce constat sans modifier son seuil après coup.

Partie 3 : Rechercher une nouvelle méthode

a. Choisissez une méthode de détection d'anomalies absente du cours et des modèles imposés dans ce TP. Une implémentation existante est autorisée ; votre travail doit démontrer la compréhension de la méthode.

b. Citez une publication ou une documentation officielle. Expliquez le principe, les hypothèses et la construction du score.

c. Justifiez l'intérêt attendu sur Credit Card et identifiez un ou deux hyperparamètres importants.

d. Intégrez la méthode aux mêmes fonctions d'évaluation, avec les mêmes partitions et la même règle de seuil. Limitez-vous à trois configurations.

e. Comparez-la à IF et LOF. Discutez un cas de réussite, une limite et le coût de calcul. Une méthode moins performante peut donner lieu à une excellente analyse.

Partie 4 : Transférer à la maintenance

a. Chargez AI4I 2020. Définissez Machine failure comme cible. Retirez des entrées les identifiants et les colonnes TWF, HDF, PWF, OSF et RNF, qui décrivent des types de panne. Ces colonnes révéleraient une information directement liée à la cible.

b. Examinez la fréquence des pannes. Traitez la variable catégorielle Type et les variables numériques avec un prétraitement ajusté uniquement sur l'apprentissage. Rappelez que les observations sont synthétiques.

c. Réutilisez Isolation Forest et le meilleur modèle supervisé sélectionné sur la validation de Credit Card. Conservez leurs hyperparamètres pour limiter le travail, mais réajustez les modèles et les transformations sur AI4I. N'imposez pas la même valeur numérique du seuil d'un jeu à l'autre.

d. Sur la validation AI4I, choisissez le seuil maximisant le rappel sous précision ≥ 90 %, puis évaluez le test. Fournissez les mêmes mesures et les nombres d'interventions inutiles et de pannes manquées. Si l'objectif est inaccessible, appliquez la règle commune de signalement.

e. Expliquez pourquoi la méthode gagnante sur les fraudes peut ne pas être la meilleure sur les pannes. Ne comparez pas directement les AP des deux jeux comme s'ils présentaient la même difficulté ou la même proportion de positifs.

Partie 5 : Robustesse et explicabilité

1. **Contaminer la référence normale**

Reprenez l'apprentissage normal de Mouse de la partie 1. Constituez des apprentissages de même taille avec 0 %, 5 % et 10 % d'anomalies synthétiques, en remplaçant des observations normales. Précisez comment vous générez ces anomalies et conservez exactement le même test réel réservé.

Réajustez IF et LOF avec des hyperparamètres fixes. Répétez sur trois graines et représentez la moyenne et la dispersion de l'AP en fonction de la contamination. N'injectez rien dans le test. Expliquez les différences observées et la dépendance des conclusions au générateur synthétique choisi.

2. Expliquer quelques alertes

Sur Credit Card, expliquez avec SHAP les sorties d'Isolation Forest pour six observations de test : autant que possible, deux fraudes détectées, deux fausses alertes et deux fraudes manquées. Si une catégorie est absente, signalez-le et complétez avec d'autres cas.

Utilisez un petit échantillon de référence issu uniquement de l'apprentissage, par exemple 50 observations. Identifiez précisément la sortie expliquée : certains outils expliquent une grandeur interne liée aux chemins plutôt que le score final. Vérifiez le sens des contributions avant de conclure.

Comparez les variables qui contribuent aux alertes. Expliquez ce que SHAP permet de décrire et ce qu'il ne permet pas d'affirmer : une contribution au modèle n'est pas une preuve de causalité ni une signification métier retrouvée pour une variable anonymisée.

Livrables et synthèse

Remettez le notebook exécuté, utils.py, les dépendances et une synthèse de deux pages maximum. Présentez votre protocole, les résultats principaux, la méthode recherchée et ses sources, les contraintes métier atteintes ou non, ainsi que les limites de généralisation. Conservez les analyses détaillées et les figures dans le notebook.

La qualité du travail repose sur la rigueur du protocole, la reproductibilité et l'interprétation. Un score élevé obtenu avec une fuite de données ne constitue pas un résultat valide.

1. Sur la base de données Mouse

Ce fichier contient 500 instances décrites par deux variables $\mathbf{x_1}$ et $\mathbf{x_2}$, représentant des points de la tête de Mickey Mouse. Les 10 dernières instances du fichier sont aberrantes (*outliers*).

- Télécharger ce jeu de données et analyser le.
- Donner une représentation graphique des données.
- Appliquer la technique **Isolation Forest** pour détecter les outliers dans ce jeu de données.
- Appliquer la technique **Local Outlier Factor** pour détecter les outliers dans ce jeu de données.
- Modifier la représentation graphique précédente pour visualiser les données aberrantes avec chacune de ces deux approches.
- Afficher **l'histogramme des scores d'anomalie** pour chacune des deux approches (à l'aide de ***seaborn*** ou ***matplotlib***)
- Pour chacune de ces deux approches, proposer deux méthodologies non supervisée (***clustering*** et ***écart interquartile*** **IQR**) permettant de mieux choisir le seuil de décision (anomalie ou non).
- Comparer les résultats obtenus numériquement et visuellement.
- Proposer une méthodologie pour la **détection de nouveautés** utilisant l'approche **Local Outlier**

Factor[1] et analyser visuellement les résultats obtenus.

2. Sur le jeu de données des cartes de crédits et de détection d'intrusions dans les réseaux

L'objectif au début de cette partie est de **détecter les fraudes dans les transactions de cartes bancaires**. Pour plus d'informations sur ce jeu de données, vous pouvez visiter le lien suivant (https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud).

Ce jeu de données est très déséquilibré avec seulement 0.172% de fraudes (492 fraudes sur 284 807 transactions). Si vous rencontrez des problèmes de performance, vous pouvez utiliser un **échantillon aléatoire stratifié** de ce jeu de données. Sinon, vous pouvez travailler sur la totalité des données.

a) Préparation des données

Préparer ce jeu de données (ne pas utiliser la variable Time).

b) Méthodologie de détection d'outliers

Proposer une méthodologie pour **la détection d'outliers** en comparant plusieurs approches entre elles :

- **Deux approches supervisées classiques**, combinées à des techniques de gestion du déséquilibre des classes :
  - *Undersampling* (ex. : **Tomek Links**, disponible dans *imblearn*),
  - *Oversampling* (ex. : **SMOTE**, disponible dans *imblearn*),
  - et *balancing* (rééquilibrage global).
- **L'approche EasyEnsemble** (disponible dans *imblearn*).
- **L'approche RUSBoostClassifier** (disponible dans *imblearn*).
-
- **Un modèle XGBoost**.
- **Deux approches non supervisées** : **Isolation Forest** et **Local Outlier Factor (LOF)**.

Points à prendre en compte dans votre méthodologie :

- Utiliser un **protocole de comparaison rigoureux** :
  - soit basé sur un **échantillonnage aléatoire stratifié**,
  - soit sur une **validation croisée stratifiée** (si les performances computationnelles le permettent).
- **Évaluer les performances** à l'aide de métriques appropriées (voir le cours sur Moodle) :
  - Matrice de confusion,
  - *F1-score* ou *Balanced Accuracy*,
  - *ROC AUC* ou *Average Precision Score*.
- **Afficher les courbes de performance** :
  - *ROC Curve* et/ou *Precision-Recall Curve*,
  - Indiquer le *Recall@Precision ≥ X*.
- **Choisir le seuil de décision optimal pour obtenir votre meilleur score**
- **Normaliser les données** si nécessaire : StandardScaler, MinMaxScaler, ou RobustScaler. Bien choisir la bonne méthodologie.

[1] https://scikit-learn.org/stable/modules/outlier_detection.html#outlier-detection

- **Encoder les variables catégorielles** si besoin : OneHotEncoder ou OrdinalEncoder.
- **Optimiser les hyperparamètres** : Par exemple :
  - Pour Isolation Forest → nombre d'arbres (*n_estimators*),
  - Pour LOF → nombre de voisins (*n_neighbors*),
- **Factoriser votre code** en fonctions modulaires pour automatiser la comparaison entre méthodes.

c) Application sur le jeu de données KDDCup99

Appliquer la même méthodologie à un second jeu de données : la **détection d'intrusions dans les réseaux** (KDDCup99).

3. Bonus (pour aller plus loin) :

- **Explicabilité** : proposer une approche d'interprétation des résultats d'**Isolation Forest sur** les **données Credit Card**, permettant d'identifier les **variables les plus importantes** dans la détection d'anomalies (via la bibliothèque **SHAP**).
- **Robustesse** : évaluer la robustesse des différentes approches par **injection d'anomalies synthétiques**, et **afficher graphiquement les scores de performance** en fonction du **taux d'anomalies injectées**.
- **Détection de nouveautés** : proposer une **méthodologie basée sur un *autoencodeur simple*** pour la **détection de nouveautés** sur les **données Mouse**.